Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Розробка графічного редактора об’єктів»

Виконав:

студент групи ІМ-о51
Мосендз Белла
Номер в списку - 9

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту:

- динамічний масив для Shape обсягом 109 об’єктів
- "гумовий" слід – суцільна лінія червоного кольору
- прямокутник:
- ввід від центру до одного з кутів
- чорний контур прямокутника без заповнення
- еліпс:
- по двом протилежним кутам охопл. прямокутника
- чорний контур з кольоровим заповненням
- колір заповнення: блакитний
- позначка поточного типу об’єкту: в заголовку вікна

Main file: MainWindow.xaml.cs

```
using System.Windows;

using System.Windows.Input;

using Lab2.Models;



namespace Lab2;



public partial class MainWindow : Window

{

    private DrawingTool _currentTool = DrawingTool.Point;

    private Point? _dragStart;

    private UIElement? _previewShape;



    // не більше 109 ел в масиві

    private const int MaxShapes = 109;

    private readonly Shape?[] _shapes = new Shape?[MaxShapes];

    private int _shapeCount = 0;



    private void SelectTool(DrawingTool tool)

    {

        _currentTool = tool;

        // повертає значення, не виконує код
```

```
        string toolName = tool switch

        {

            DrawingTool.Point => "Крапка",

            DrawingTool.Line => "Лінія",

            DrawingTool.Rectangle => "Прямокутник",

            DrawingTool.Ellipse => "Еліпс",

            _ => "Невідомий інструмент"

        };



        Title = $"Lab2 - {toolName}";

    }

    private void DrawingCanvas_MouseLeftButtonDown(object sender,
MouseButtonEventArgs e)

    {

        if (_shapeCount >= MaxShapes)

        {

            MessageBox.Show(

                $"Полотно заповнене. Максимальна кількість фігур —
{MaxShapes}.",

                "Обмеження",

                MessageBoxButton.OK,

                MessageBoxImage.Information
```

```
            );
            return;
        }
        Point position = e.GetPosition(drawingCanvas);
        if (_currentTool == DrawingTool.Point)
        {
            Shape point = new PointShape(position);
            AddShape(point);
            return;
        }

        if (_currentTool == DrawingTool.Line ||
            _currentTool == DrawingTool.Rectangle ||
            _currentTool == DrawingTool.Ellipse)
        {
            _dragStart = position;
            drawingCanvas.CaptureMouse();
        }
    }

    private void DrawingCanvas_MouseMove(object sender, MouseEventArgs e)
```

```
    {
        if (_dragStart is not Point start)
        {
            return;
        }

        Point end = e.GetPosition(drawingCanvas);
        Shape? shape = CreateDraggedShape(start, end);

        if (shape == null)
        {
            return;
        }

        if (_previewShape != null)
        {
            drawingCanvas.Children.Remove(_previewShape);
        }

        _previewShape = shape.CreateVisual(true);
        drawingCanvas.Children.Add(_previewShape);
```

```
    }

    private void DrawingCanvas_MouseLeftButtonUp(object sender,
MouseButtonEventArgs e)
    {
        if (_dragStart is not Point start)
        {
            return;
        }
        Point end = e.GetPosition(drawingCanvas);

        if (_previewShape != null)
        {
            drawingCanvas.Children.Remove(_previewShape);
            _previewShape = null;
        }

        if ((end - start).Length >= 2)
        {
            Shape? shape = CreateDraggedShape(start, end);
            if (shape != null)
            {
```

```
                AddShape(shape);

            }

        }

        _dragStart = null;

        drawingCanvas.ReleaseMouseCapture();

    }

    private Shape? CreateDraggedShape(Point start, Point end)

    {

        return _currentTool switch

        {

            DrawingTool.Line => new LineShape(start, end),

            DrawingTool.Rectangle => new RectangleShape(start, end),

            DrawingTool.Ellipse => new EllipseShape(start, end),

            _ => null

        };

    }



    private void AddShape(Shape shape)

    {

        if (_shapeCount >= MaxShapes)

        {
```

```
            MessageBox.Show(
                "Досягнуто максимальної кількості фігур (109).",
                "Обмеження",
                MessageBoxButton.OK,
                MessageBoxImage.Information
            );
            return;
        }

        _shapes[_shapeCount] = shape;
        _shapeCount++;
        drawingCanvas.Children.Add(shape.CreateVisual(false));

        UpdateShapeCount();
    }

    private void UpdateShapeCount()
    {
        shapeCountText.Text = $"Фігур: {_shapeCount} / {MaxShapes}";
    }
    public MainWindow()
```

```
    {
        InitializeComponent();
        SelectTool(DrawingTool.Point);
        UpdateShapeCount();
    }

    private void Point_Click(object sender, RoutedEventArgs e)
    {
        SelectTool(DrawingTool.Point);
    }
    private void Line_Click(object sender, RoutedEventArgs e)
    {
        SelectTool(DrawingTool.Line);
    }
    private void Rectangle_Click(object sender, RoutedEventArgs e)
    {
        SelectTool(DrawingTool.Rectangle);
    }
    private void Ellipse_Click(object sender, RoutedEventArgs e)
    {
        SelectTool(DrawingTool.Ellipse);
```

```
    }

}
```

DrawingTool.cs

```
namespace Lab2;



public enum DrawingTool

{

    Point,

    Line,

    Rectangle,

    Ellipse

}
```

Shape.cs

```
using System.Windows;



namespace Lab2.Models;



// абстрактний клас (неможливо створити екземпляр, призначений лише для
наслідування)
```

```
// далі override у різних дочірніх класів - поліморфізм

internal abstract class Shape

{

    protected Shape(Point start, Point end)

    {

        Start = start;

        End = end;

    }



    public Point Start { get; }

    public Point End { get; }



    public abstract UIElement CreateVisual(bool isPreview);

}
```

PointShape.cs

```
using System.Windows;

using System.Windows.Controls;

using System.Windows.Media;

using System.Windows.Shapes;
```

```
namespace Lab2.Models;



internal class PointShape : Shape

{

    public PointShape(Point position) : base(position, position)

    {



    }



    public override UIElement CreateVisual(bool isPreview)

    {

        const double diameter = 6;



        var dot = new Ellipse

        {

            Width = diameter,

            Height = diameter,

            Fill = isPreview ? Brushes.Red : Brushes.Black

        };



        Canvas.SetLeft(dot, Start.X - diameter / 2);
```

```
        Canvas.SetTop(dot, Start.Y - diameter / 2);



        return dot;

    }

}
```

LineShape.cs

```
using System.Windows;

using System.Windows.Media;

using System.Windows.Shapes;



namespace Lab2.Models;



internal class LineShape : Shape

{

    public LineShape(Point start, Point end) : base(start, end)

    {

    }



    public override UIElement CreateVisual(bool isPreview)

    {
```

```
        var line = new Line
        {
            X1 = Start.X,
            Y1 = Start.Y,
            X2 = End.X,
            Y2 = End.Y,

            Stroke = isPreview ? Brushes.Red : Brushes.Black,
            StrokeThickness = 2,
            IsHitTestVisible = false
        };

        return line;
    }
}
```

RectangleShape.cs

```
using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
```

```
using System.Windows.Shapes;



namespace Lab2.Models;



internal class RectangleShape : Shape

{

    public RectangleShape(Point center, Point corner)

        : base(center, corner)

    {

    }



    public override UIElement CreateVisual(bool isPreview)

    {

        double halfWidth = Math.Abs(End.X - Start.X);

        double halfHeight = Math.Abs(End.Y - Start.Y);



        var rectangle = new Rectangle

        {

            Width = halfWidth * 2,

            Height = halfHeight * 2,

            Stroke = isPreview ? Brushes.Red : Brushes.Black,
```

```
            StrokeThickness = 2,
            Fill = null,
            IsHitTestVisible = false
        };

        Canvas.SetLeft(rectangle, Start.X - halfWidth);
        Canvas.SetTop(rectangle, Start.Y - halfHeight);

        return rectangle;
    }
}
```

EllipseShape.cs

```
using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab2.Models;

internal class EllipseShape : Shape
```

```
{
    public EllipseShape(Point start, Point end)
        : base(start, end)
    {
    }

    public override UIElement CreateVisual(bool isPreview)
    {
        double left = Math.Min(Start.X, End.X);
        double top = Math.Min(Start.Y, End.Y);

        double width = Math.Abs(End.X - Start.X);
        double height = Math.Abs(End.Y - Start.Y);

        var ellipse = new Ellipse
        {
            Width = width,
            Height = height,
            Stroke = isPreview ? Brushes.Red : Brushes.Black,
            StrokeThickness = 2,
            Fill = isPreview ? null : Brushes.SkyBlue,
```

```
            IsHitTestVisible = false

        };



        Canvas.SetLeft(ellipse, left);

        Canvas.SetTop(ellipse, top);

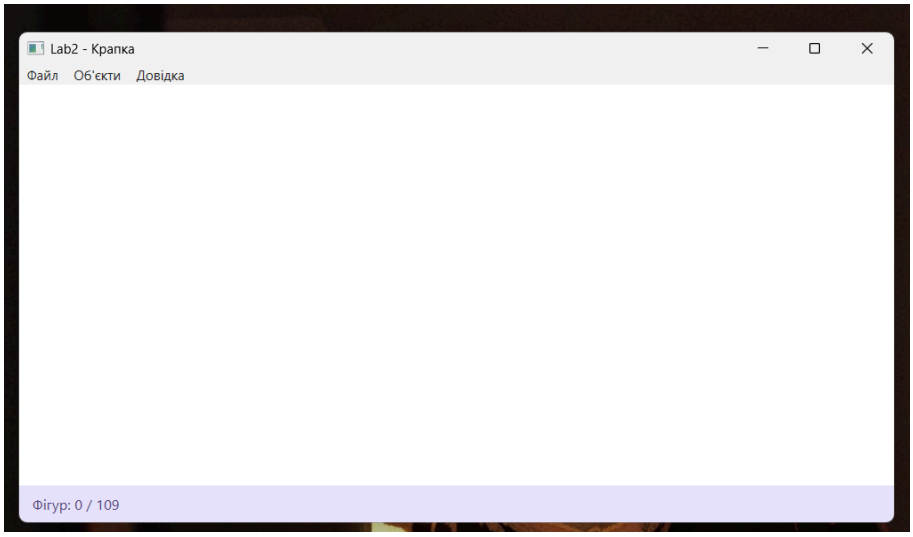


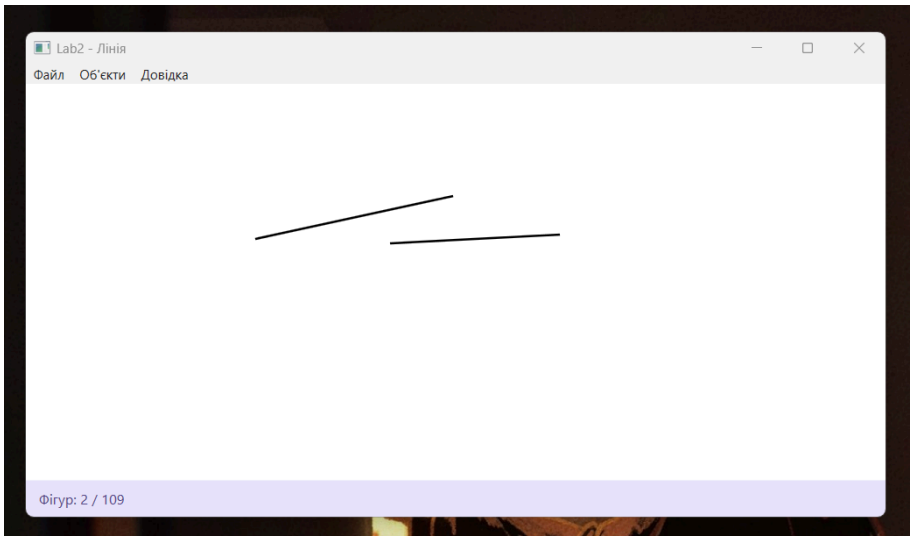
        return ellipse;

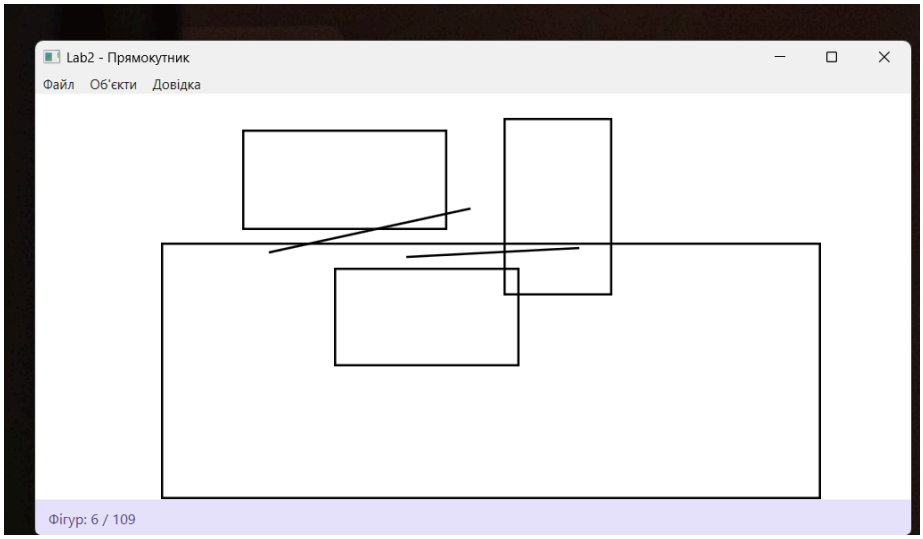
    }

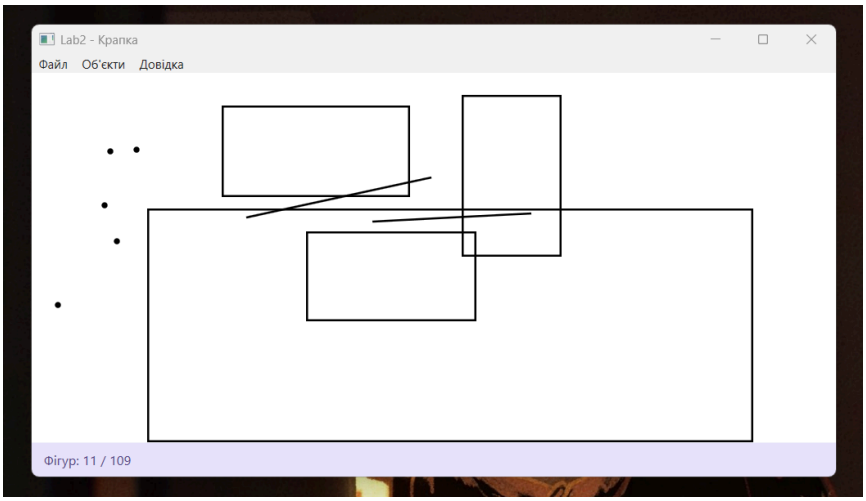
}
```

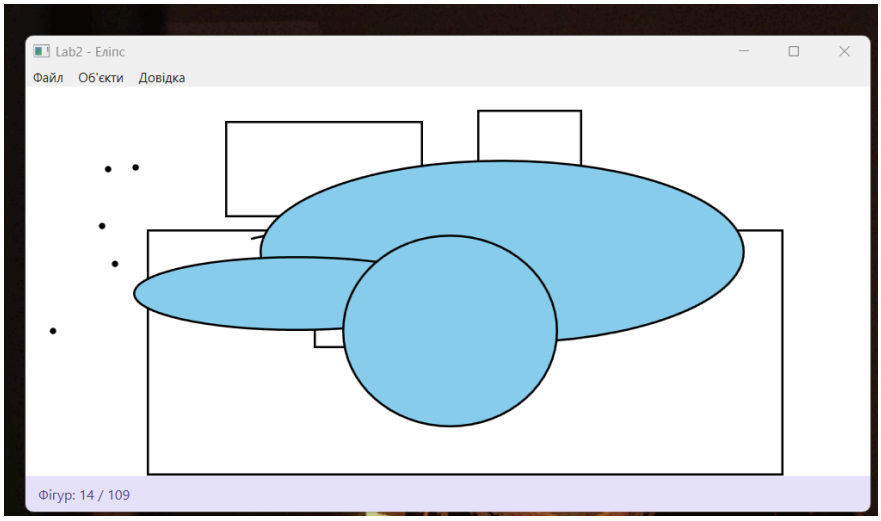
Скріншоти


Lab2 - Крапка
Файл Об'єкти Довідка
Фігур: 0 / 109


Lab2 - Лінія
Файл Об'єкти Довідка
Фігур: 2 / 109


Lab2 - Прямокутник
Файл Об'єкти Довідка
Фігур: 6 / 109


Lab2 - Крапка
Файл Об'єкти Довідка
Фігур: 11 / 109


Lab2 - Еліпс
Файл Об'єкти Довідка
Фігур: 14 / 109

Діаграма залежностей файлів та модулів

```
Lab2.csproj
├── App.xaml
├── App.xaml.cs
├── MainWindow.xaml
├── MainWindow.xaml.cs
├── DrawingTool.cs
└── Models/
    ├── Shape.cs
    ├── PointShape.cs
    ├── LineShape.cs
    ├── RectangleShape.cs
    └── EllipseShape.cs
```

```
App.xaml
├── App.xaml.cs
└── MainWindow.xaml (StartupUri)
```

```
MainWindow.xaml
├── MainWindow.xaml.cs
└── System.Windows.Controls
    ├── DockPanel
    ├── Menu
    ├── MenuItem
    ├── Canvas
    ├── Border
    └── TextBlock
```

```
MainWindow.xaml.cs
```

```
├── System.Windows
├── System.Windows.Input
├── DrawingTool.cs
│   └── DrawingTool (enum)
└── Models/
    ├── Shape.cs
    │   └── Shape (abstract class)
    ├── PointShape.cs
    │   └── PointShape
    ├── LineShape.cs
    │   └── LineShape
    ├── RectangleShape.cs
    │   └── RectangleShape
    └── EllipseShape.cs
        └── EllipseShape
```

```
Models/Shape.cs
└── System.Windows
    ├── Point
    └── UIElement
```

```
Models/PointShape.cs
├── Shape.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── System.Windows.Media
│   └── Brushes
└── System.Windows.Shapes
    └── Ellipse
```

Models/LineShape.cs

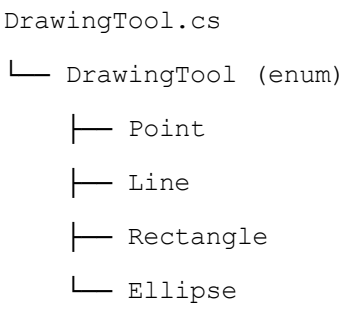
```
├── Shape.cs
├── System.Windows
├── System.Windows.Media
│   └── Brushes
└── System.Windows.Shapes
    └── Line
```

Models/RectangleShape.cs

```
├── Shape.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── System.Windows.Media
│   └── Brushes
└── System.Windows.Shapes
    └── Rectangle
```

Models/EllipseShape.cs

```
├── Shape.cs
├── System.Windows
├── System.Windows.Controls
│   └── Canvas
├── System.Windows.Media
│   └── Brushes
└── System.Windows.Shapes
    └── Ellipse
```


DrawingTool.cs
DrawingTool (enum)
Point
Line
Rectangle
Ellipse

Висновки:

Під час виконання лабораторної роботи було отримано практичні навички застосування основних принципів об’єктно-орієнтованого програмування: інкапсуляції, абстракції, успадкування та поліморфізму. Було розроблено простий графічний редактор мовою C# із використанням технології WPF, який дозволяє створювати та відображати геометричні фігури. Реалізовано ієрархію класів на основі абстрактного класу Shape, обробку подій миші та зберігання графічних об’єктів у масиві. У результаті закріплено навички проєктування класів, організації взаємодії між об’єктами та розробки програм в об’єктно-орієнтованому стилі.